

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

HERRAMIENTAS QUE GENERAN CÓDIGO FUENTE A PARTIR DE MODELOS UML (CICLO MDD)

ASIGNATURA:

INGENIERÍA DE REQUISITOS

DOCENTE:

ING. GLEISTON GUERRERO ULLOA, PhD.

INTEGRANTES:

CHAVARRIA CUENCA TAHINY MEL, CI: 0943050054, tchavarriac@uteq.edu.ec

MENDOZA PÁRRAGA ANDY JOHEL, CI: 1251401590, amendozap9@uteq.edu.ec, Titular PFC

SANCHEZ CORNEJO GARY ALBERTO, CI: 1208338291, gsanchezc6@uteq.edu.ec

EQUIPO:

H

CURSO/PARALELO:

CUARTO SEMESTRE PARALELO "B"

SISTEMA PFC:

MundiPets (Titular: Andy Mendoza Párraga)

HERRAMIENTA MDD SELECCIONADA:

BOUML

REPOSITORIO GITHUB:

<https://github.com/andymendozap03/PFC-MundiPets-MDD-EquipoH-BOUML.git>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

Resumen	3
1. Introducción	4
2. Marco teórico	5
2.1. Model-Driven Engineering (MDE)	5
2.2. Model-Driven Architecture (MDA)	5
2.3. Model-Driven Development (MDD)	5
2.4. Transformaciones M2M	5
2.5. Transformaciones M2T	5
2.6. Metamodelo	5
2.7. Perfiles UML	5
2.8. Plantilla / template	6
2.9. Forward, reverse y round-trip engineering	6
2.10. Diagramas UML admisibles como origen	6
2.11. Estado del arte	6
3. Justificación de la selección de BOUML	6
3.1. Herramienta seleccionada y versión	6
3.2. Justificación frente a los requisitos del PFC	6
3.3. Comparación con alternativa descartada	6
4. Descripción del subsistema del PFC modelado	6
4.1. Contexto de MundiPets	6
4.2. Módulo seleccionado	6
4.3. Actores	6
4.4. Requisitos funcionales	6
5. Modelo UML de origen	6
5.1. Diagrama de clases	6
5.2. Relaciones UML	7
5.3. Estereotipos	8
5.4. Atributos y operaciones tipificados	8
5.5. Diagramas complementarios	8
5.6. Validación del modelo en BOUML	9
6. Configuración del generador	11
6.1. Plantilla y parámetros	11
6.2. Política de regeneración y decisiones de mapeo	11
7. Proceso de generación	11
7.1. Ejecución y capturas	11
7.2. Árbol de proyecto y trazabilidad clase → archivo	11
8. Verificación del código generado	11
8.1. Compilación y ejecución	11
8.2. Correspondencia modelo → código	11
8.3. Limitaciones detectadas	11
9. Cierre del ciclo (roundtrip)	11
9.1. Cambio aplicado al modelo	11
9.2. Evidencia antes/después	11
10. Reparto del trabajo	11
11. Conclusiones	11
Referencias	12

Anexos

13

Resumen

1. Introducción

2. Marco teórico

2.1. Model-Driven Engineering (MDE)

2.2. Model-Driven Architecture (MDA)

La iniciativa Model-Driven Architecture del Object Management Group estructura el desarrollo de software en tres niveles de abstracción [1]:

- **CIM** (*Computation Independent Model*): describe el dominio y el negocio del sistema sin referencia a aspectos computacionales, capturando los requisitos desde la perspectiva de quienes conocen el negocio.
- **PIM** (*Platform Independent Model*): expresa la lógica y estructura del sistema con independencia de la plataforma de implementación, constituyendo el modelo de diseño.
- **PSM** (*Platform Specific Model*): vincula el modelo a una plataforma de despliegue concreta (por ejemplo, un lenguaje de programación o un framework), añadiendo los detalles técnicos necesarios para la generación de código.

Entre estos niveles median transformaciones formales que permiten derivar un modelo del siguiente sin reescribirlo manualmente [1]. Si bien la mayoría de herramientas y estudios se concentran en la transformación PIM–PSM, la transformación CIM–PIM recibe comparativamente menos atención en la literatura especializada, pese a ser la que conecta los requisitos de negocio con el primer modelo de diseño independiente de plataforma [2].

2.3. Model-Driven Development (MDD)

2.4. Transformaciones M2M

2.5. Transformaciones M2T

2.6. Metamodelo

UML se define formalmente sobre MOF, el estándar OMG que especifica la sintaxis abstracta: clases, atributos, asociaciones, generalizaciones y estereotipos, que todo modelo UML válido debe respetar [3]. Un generador de código no interpreta el diagrama libremente: aplica reglas de transformación basadas en los tipos de elementos que el metamodelo garantiza que existen [4]. La calidad del metamodelo de origen condiciona directamente la del artefacto generado: especificar incorrectamente entidades o atributos de un metamodelo impacta negativamente en la calidad de los artefactos derivados de él [5], lo que confirma por qué la validación del modelo de origen (Sección 5.5) es un paso crítico y no un formalismo opcional. BOUML recorre las clases, atributos, operaciones y relaciones conformes al metamodelo UML del proyecto .prj para producir los pares .h/ .cpp: cada `uml:Class` genera un par de archivos, cada `uml:Attribute` tipado un campo privado, y cada `uml:Generalization` herencia pública de C++

2.7. Perfiles UML

Los perfiles UML son el mecanismo estándar de extensión ligera del metamodelo: mediante estereotipos, valores etiquetados y restricciones, adaptan el vocabulario genérico de UML a un dominio o plataforma concreta (p. ej., un perfil «JPA» o «Spring») sin modificar la especificación base [3]. Un estereotipo se aplica sobre una metaclassa existente (`Class`, `Action`) y añade semántica que el generador de código explota para decidir qué construcción del lenguaje objetivo producir. Trabajos recientes confirman la vigencia y el rigor exigido a este mecanismo: Górski [6] define un perfil completo de patrones de mensajería para microservicios e IoT mediante estereotipos y valores etiquetados reutilizables entre proyectos, mientras que Kraas [7] demuestra que un perfil bien construido debe preservar mediante OCL las restricciones de buena formación del metamodelo de origen, y no limitarse a una lista decorativa de nombres.

BOUML no ofrece un editor de perfiles formal en sentido estricto, pero permite aplicar estereotipos directamente sobre las clases del modelo con la misma función práctica: se aplicó «Entity» a las clases persistentes del subsistema (*Mascota*, *HistorialMedico*, *Vacuna*, *MascotaVacuna*, *EvaluacionVeterinaria*) y «Actor» a la jerarquía de *Usuario*, de modo que el generador C++ distingue en tiempo de generación las clases de dominio persistentes de las clases de control.

2.8. Plantilla / template

2.9. Forward, reverse y round-trip engineering

2.10. Diagramas UML admisibles como origen

El diagrama de clases constituye la base de la generación estructural de código, mientras que los diagramas de casos de uso, estados y secuencia cumplen un rol complementario cuando aportan contexto o comportamiento relevante al generador. Esta jerarquía coincide con la evidencia empírica: en una encuesta a 109 profesionales de 34 países, el diagrama de clases resultó ser la primera opción de los practicantes para modelar tanto la estructura funcional (71 %) como la de datos (85 %) de un sistema, muy por encima de otros diagramas UML [8]. Esto confirma que centrar la generación de código en el diagrama de clases no es una simplificación arbitraria, sino la práctica dominante en la industria.

El subsistema de gestión sanitaria de MundiPets se modeló exclusivamente mediante diagrama de clases, sin diagramas complementarios: el alcance definido es puramente estructural: clases, atributos, relaciones y herencia, y no involucra comportamiento complejo (máquinas de estado) ni interacciones críticas entre objetos que ameriten un diagrama de secuencia, por lo que agregar diagramas adicionales habría sido relleno sin aporte real a la generación de código en BOUML.

2.11. Estado del arte

3. Justificación de la selección de BOUML

3.1. Herramienta seleccionada y versión

3.2. Justificación frente a los requisitos del PFC

3.3. Comparación con alternativa descartada

4. Descripción del subsistema del PFC modelado

4.1. Contexto de MundiPets

4.2. Módulo seleccionado

4.3. Actores

4.4. Requisitos funcionales

5. Modelo UML de origen

En esta sección se detalla la estructuración, las características técnicas y las restricciones semánticas del modelo UML desarrollado directamente dentro del entorno operativo de BOUML. Este modelo consolida la lógica del negocio para el subsistema de Gestión de Mascotas e Historial Médico de MundiPets, sirviendo como la fuente de datos autoritativa y estructurada para el proceso posterior de ingeniería directa y generación automática de código fuente.

5.1. Diagrama de clases

El diagrama de clases estructural del subsistema sanitario se construyó y normalizó de manera nativa en BOUML, adaptando las especificaciones del dominio a la sintaxis del metamodelo. En la Figura 1 se expone el lienzo de trabajo final, el cual integra la totalidad de los compartimentos de atributos, operaciones y visibilidades requeridos para la correcta compilación del sistema.

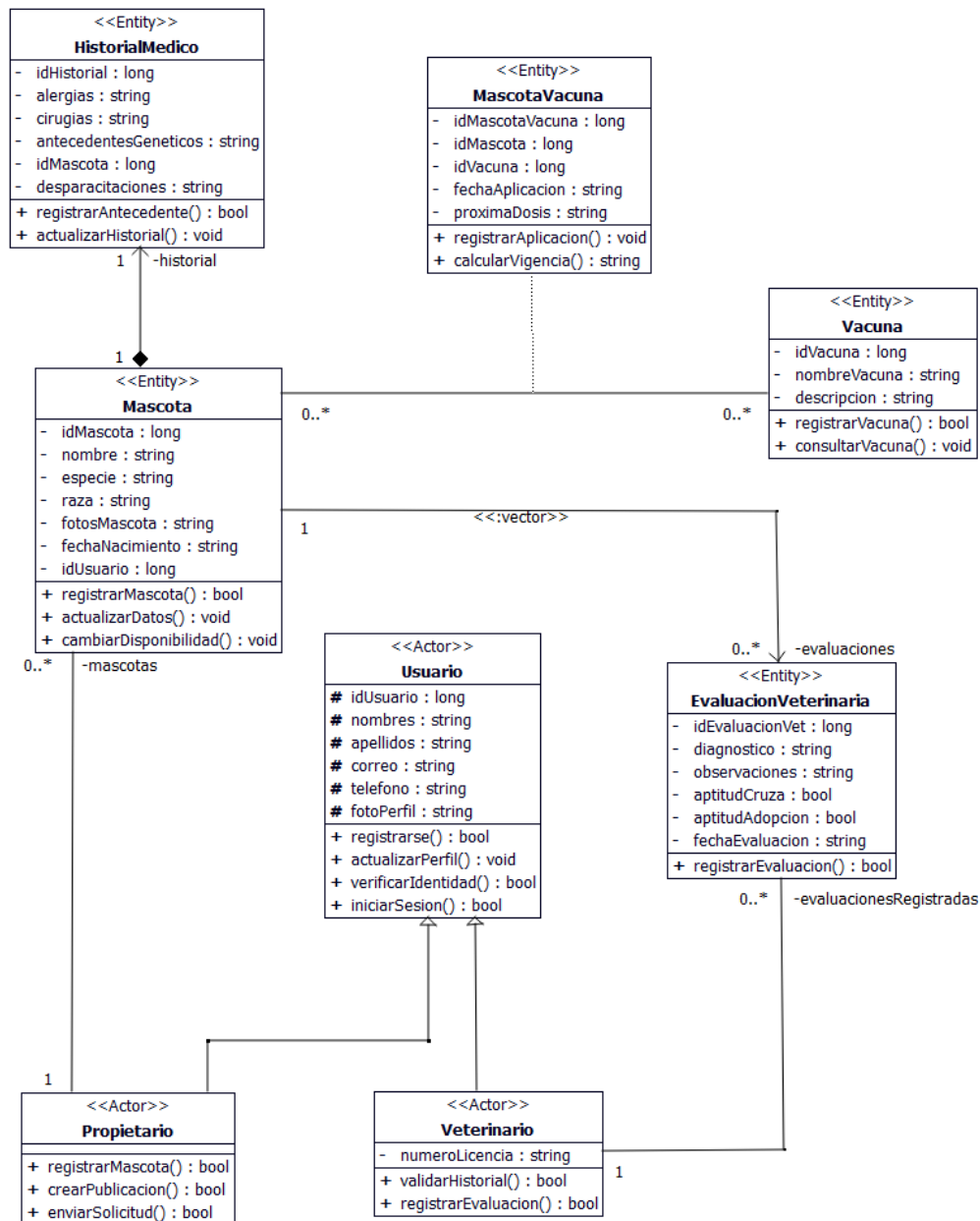


Figura 1: Diagrama de clases del subsistema sanitario refinado en el entorno de BOUML.

5.2. Relaciones UML

Las conexiones que trazamos en el lienzo de BOUML dictan las reglas del negocio y definen cómo se van a comportar los punteros y contenedores en el código de C++. En la Tabla 1 desglosamos detalladamente qué significa cada multiplicidad y tipo de enlazado en nuestro diseño.

Tabla 1: Especificación de relaciones y multiplicidades del modelo en BOUML.

Clases Involucradas	Tipo de Relación	Descripción de la Restricción UML
<i>Usuario</i> $\leftarrow$ <i>Propietario</i>	Generalización	Herencia pura. La clase <i>Propietario</i> hereda y especializa los atributos y comportamiento del <i>Usuario</i> base.
<i>Usuario</i> $\leftarrow$ <i>Veterinario</i>	Generalización	Herencia pura. La clase <i>Veterinario</i> extiende al <i>Usuario</i> común, añadiendo su número de licencia profesional.
<i>Mascota</i> $\blacklozenge \rightarrow$ <i>HistorialMedico</i>	Composición	Relación de existencia fuerte (1:1). El <i>HistorialMedico</i> depende totalmente de que exista la <i>Mascota</i> ; si esta se elimina, su historial también.
<i>Propietario</i> (1) $\leftrightarrow$ <i>Mascota</i> (0..*)	Asociación Simple	Navegabilidad que define la propiedad de la mascota. Un dueño puede administrar una colección de cero o muchas mascotas en su cuenta.
<i>Mascota</i> (*) $\leftrightarrow$ <i>Vacuna</i> (*)	Clase Asociativa	Relación muchos a muchos (M:N) del estándar UML, mapeando que múltiples mascotas reciben diferentes vacunas del catálogo.
<i>Mascota</i> (1) $\leftrightarrow$ <i>MascotaVacuna</i> (*)	Vínculo de Instancia	Conexión estructural que une a una mascota específica con cada una de las dosis inyectadas en su carné sanitario.
<i>Vacuna</i> (1) $\leftrightarrow$ <i>MascotaVacuna</i> (*)	Vínculo de Catálogo	Conexión que asocia el registro de la dosis aplicada con las especificaciones y nombre de la vacuna biológica base.
<i>Mascota</i> (1) $\rightarrow$ <i>EvaluacionVeterinaria</i> (0..*)	Asociación Unidireccional	Define el flujo de revisiones. Una mascota acumula a lo largo del tiempo un historial de múltiples evaluaciones clínicas mediante un vector.
<i>Veterinario</i> (1) $\leftrightarrow$ <i>EvaluacionVeterinaria</i> (0..*)	Asociación Simple	Vincula la responsabilidad clínica. Un veterinario es el encargado de firmar, auditar y registrar múltiples evaluaciones médicas en el sistema.

5.3. Estereotipos

Para que el diagrama no sea solo un montón de cajas y el generador de BOUML entienda qué función cumple cada clase en la vida real, les asignamos marcas especiales llamadas *estereotipos*. En el diseño del sistema usamos dos tipos principales que nos ayudan a separar los roles de los usuarios de los datos puros del negocio:

- **«Actor»:** Este estereotipo lo usamos para identificar a las clases que representan a personas o roles que van a interactuar directamente con las pantallas del sistema. En nuestro caso, se lo pusimos a la clase base *Usuario* y a sus herencias directas: *Propietario* (quien registra a los animalitos) y *Veterinario* (quien realiza las consultas).
- **«Entity»:** Lo aplicamos a todas las clases encargadas de almacenar y hacer persistir la información clave de la clínica veterinaria. Aquí entran *Mascota*, *HistorialMedico*, *Vacuna*, la clase puente *MascotaVacuna* y los reportes de *EvaluacionVeterinaria*. Estas clases se encargan de los datos que deben quedarse grabados en el sistema.

5.4. Atributos y operaciones tipificados

Para que la traducción del diagrama al código de C++ sea exacta y no genere errores al compilar, nos aseguramos de que cada atributo y método en BOUML tenga su tipo de dato bien definido. No dejamos ninguna variable al aire, sino que mapeamos todo según las necesidades reales del software:

- **Identificadores únicos (long):** Todas las llaves primarias del sistema (como *idMascota*, *idHistorial* o *idUsuario*) se configuraron como de tipo `long`. Esto nos garantiza espacio de almacenamiento suficiente para manejar números grandes cuando la base de datos crezca.
- **Cadenas de texto (string):** Las variables destinadas a guardar palabras (nombres, correos, razas o descripciones clínicas) se tipificaron como `string`. Cabe destacar que en los atributos del lienzo ocultamos el prefijo estándar `std::` para que el diagrama se vea limpio y apegado a las normas gráficas de UML.
- **Métodos y operaciones (bool / void):** Las funciones que realizan controles o validaciones (como *registrarMascota()* o *iniciarSesion()*) devuelven un tipo lógico `bool` para saber si el proceso tuvo éxito o no. Por otro lado, las funciones destinadas únicamente a guardar cambios o refrescar datos (como *actualizarPerfil()*) se configuraron con tipo de retorno `void`.

5.5. Diagramas complementarios

Para complementar la perspectiva estática que nos dio el diagrama de clases, es fundamental analizar el comportamiento del sistema desde el punto de vista del usuario. Mientras que las clases definen la estructura

interna, las variables y los métodos del código, los diagramas de comportamiento nos permiten entender el alcance funcional del software y qué acciones reales pueden ejecutar los actores sobre el sistema.

Para este propósito, construimos un Diagrama de Casos de Uso de manera centralizada y nativa dentro del entorno de BOUML, enfocándonos exclusivamente en el alcance transaccional del subsistema de Gestión de Mascotas e Historial Médico. En este modelo se delimitan las fronteras del módulo (representadas por el recuadro del límite del sistema) y se detalla cómo los actores disparan los flujos principales:

- **Propietario:** Interactúa directamente con el subsistema activando los casos de uso destinados al registro de sus mascotas, la configuración de la privacidad médica de los datos y la consulta del perfil médico. Asimismo, maneja el registro del historial médico, flujo que cuenta con una relación de extensión («*extend*») hacia el caso de uso *Cargar documento veterinario*, indicando que el usuario tiene la opción opcional de adjuntar archivos de soporte durante este proceso.
- **Veterinario:** Domina el flujo técnico e institucional de la clínica, teniendo acceso a la consulta del perfil médico de los pacientes y siendo el único actor encargado de disparar el caso de uso para validar la información médica en el sistema.

El desarrollo de este diagrama de comportamiento dentro de la misma herramienta de origen garantiza una coherencia absoluta en nuestro informe, demostrando que cada caso de uso del negocio cuenta con una correspondencia directa en una clase, un método y un atributo tipificado listo para soportar su ejecución en el código real de C++.

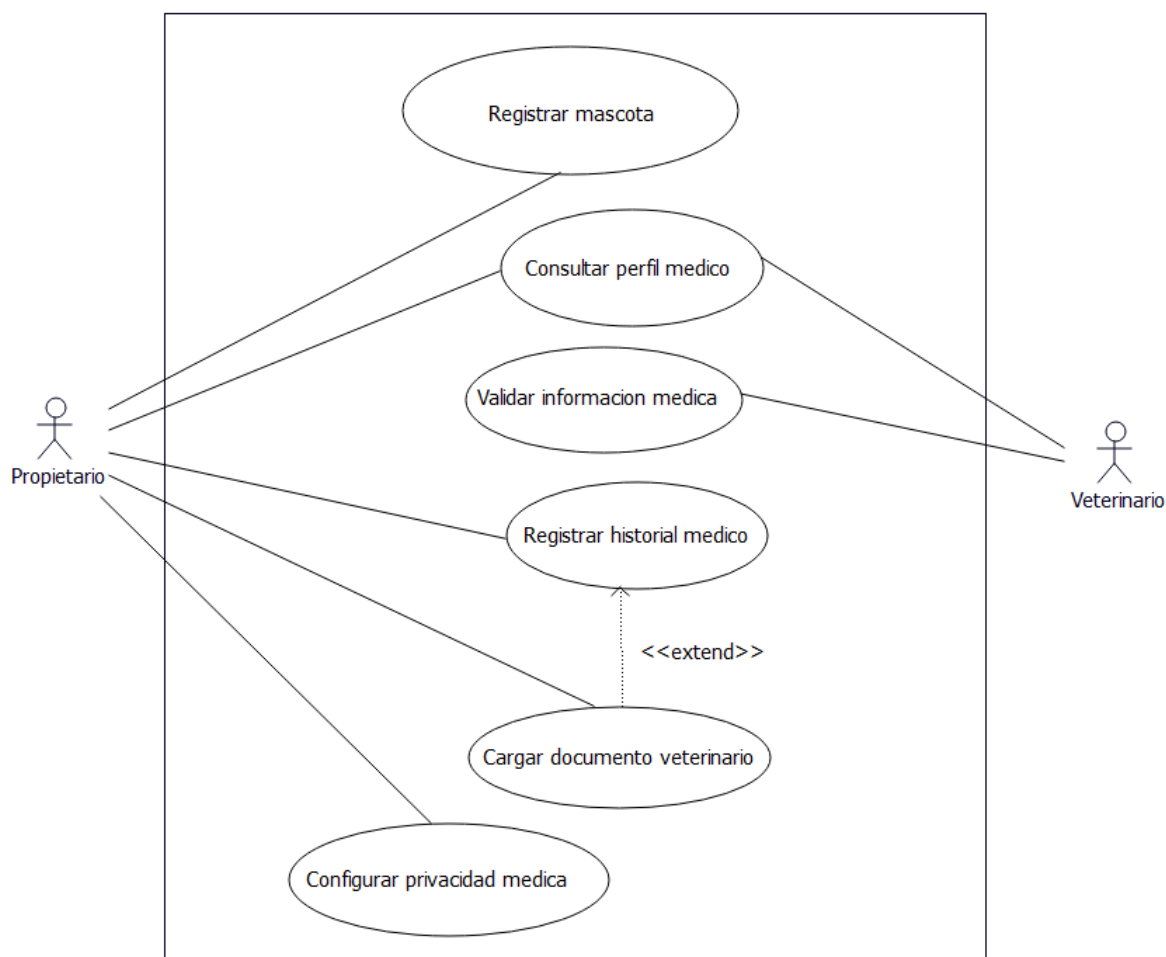


Figura 2: Diagrama de casos de uso del subsistema sanitario desarrollado nativamente en BOUML.

5.6. Validación del modelo en BOUML

Antes de dar el paso definitivo hacia la ingeniería directa y generar el código fuente en C++, sometimos todo el diseño del subsistema a la herramienta de auditoría e inspección interna de BOUML. A diferencia de otros

programas que son simples herramientas de dibujo genérico, BOUML cuenta con un analizador sintáctico que comprueba activamente que se cumplan las restricciones reales del metamodelo UML.

Para realizar esta verificación, ejecutamos el control de consistencia jerárquica sobre el árbol del proyecto mediante la herramienta *Uml projection*, monitoreando su comportamiento a través de la consola de trazas. El diagnóstico determinó que nuestro modelo está completamente limpio y libre de errores estructurales, finalizando con un estado exitoso y sin advertencias, tal como se evidencia en la Figura 3.

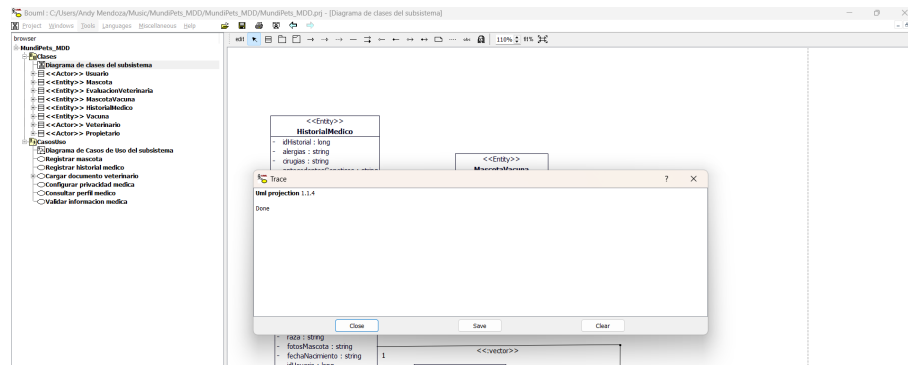


Figura 3: Consola de trazas de BOUML confirmando la validación exitosa del proyecto (Uml projection Done).

Durante este proceso automatizado, el entorno validó los siguientes puntos clave del diseño:

- **Consistencia en las relaciones:** Se comprobó que todas las asociaciones, herencias, composiciones y la clase asociativa interconectan extremos bien definidos, con multiplicidades válidas que el generador puede traducir sin ambigüedades.
- **Integridad de tipos:** Se verificó que ninguna variable o método quedara con tipos de datos "huérfanos." indefinidos, asegurando que los primitivos (string, long, bool, void) estén correctamente mapeados.
- **Validación de herencia y visibilidades:** El analizador confirmó que la jerarquía de generalización desde la clase base *Usuario* hacia *Propietario* y *Veterinario* respeta las reglas de encapsulamiento y el acceso a los atributos protegidos (#).

Gracias a este proceso de control previo, mitigamos por completo el riesgo de arrastrar errores de diseño, pérdida de tipos o inconsistencias lógicas hacia los archivos de código físico, garantizando que los archivos de cabecera y de implementación se generen de manera totalmente limpia.

6. Configuración del generador

6.1. Plantilla y parámetros

6.2. Política de regeneración y decisiones de mapeo

7. Proceso de generación

7.1. Ejecución y capturas

7.2. Árbol de proyecto y trazabilidad clase → archivo

8. Verificación del código generado

8.1. Compilación y ejecución

8.2. Correspondencia modelo → código

8.3. Limitaciones detectadas

9. Cierre del ciclo (roundtrip)

9.1. Cambio aplicado al modelo

9.2. Evidencia antes/después

10. Reparto del trabajo

11. Conclusiones

Referencias

- [1] Object Management Group, “Model Driven Architecture (MDA) Guide, Revision 2.0,” OMG, inf. téc. ormsc/14-06-01, 2014.
- [2] N. Silega et al., “Transformation From CIM to PIM: A Systematic Mapping,” *IEEE Access*, vol. 10, 2022. DOI: 10.1109/ACCESS.2022.3201556
- [3] Object Management Group, “OMG Unified Modeling Language (OMG UML), Version 2.5.1,” OMG, inf. téc. formal/17-12-05, 2017.
- [4] P. Di Felice, G. Paolone, R. Paesani y M. Marinelli, “Design and Implementation of a Metadata Repository about UML Class Diagrams. A Software Tool Supporting the Automatic Feeding of the Repository,” *Electronics*, vol. 11, n.º 2, pág. 201, 2022. DOI: 10.3390/electronics11020201
- [5] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen y A. Pierantonio, “MemoRec: A Recommender System for Assisting Modelers in Specifying Metamodels,” *Software and Systems Modeling*, vol. 22, n.º 1, págs. 203-223, 2022. DOI: 10.1007/s10270-022-00994-2
- [6] T. Górski, “UML Profile for Messaging Patterns in Service-Oriented Architecture, Microservices, and Internet of Things,” *Applied Sciences*, vol. 12, n.º 24, pág. 12 790, 2022. DOI: 10.3390/app122412790
- [7] A. Kraas, “On the Automation-Supported Derivation of Domain-Specific UML Profiles Considering Static Semantics,” *Software and Systems Modeling*, vol. 21, n.º 1, págs. 51-79, 2022. DOI: 10.1007/s10270-021-00890-1
- [8] M. Ozkaya y F. Erata, “A Survey on the Practical Use of UML for Different Software Architecture Viewpoints,” *Information and Software Technology*, vol. 121, pág. 106 275, 2020. DOI: 10.1016/j.infsof.2020.106275

Anexos